REFERENCE · TOOL SERVER OPTIONS

MCP Tool Server Options on Google Cloud: Custom, MCP Toolbox, Google-Managed

The agent should never hold a database client. The first box we add replaces `execute_sql(query)` with a small set of named operations served over the Model Context Protocol (MCP).

MCP is a protocol: it standardizes how a model discovers tools, what their input schemas are, and how calls and results are exchanged. It says nothing about what a tool is allowed to do. A tool named `run_sql(query: str)` is a valid MCP tool and exactly as dangerous as the function it replaces. The security value comes from *which* tools the server exposes, *how* it binds the caller's identity, and *what* credential it uses to reach the database. On Google Cloud there are three ways to run this layer, and they differ on all three points.

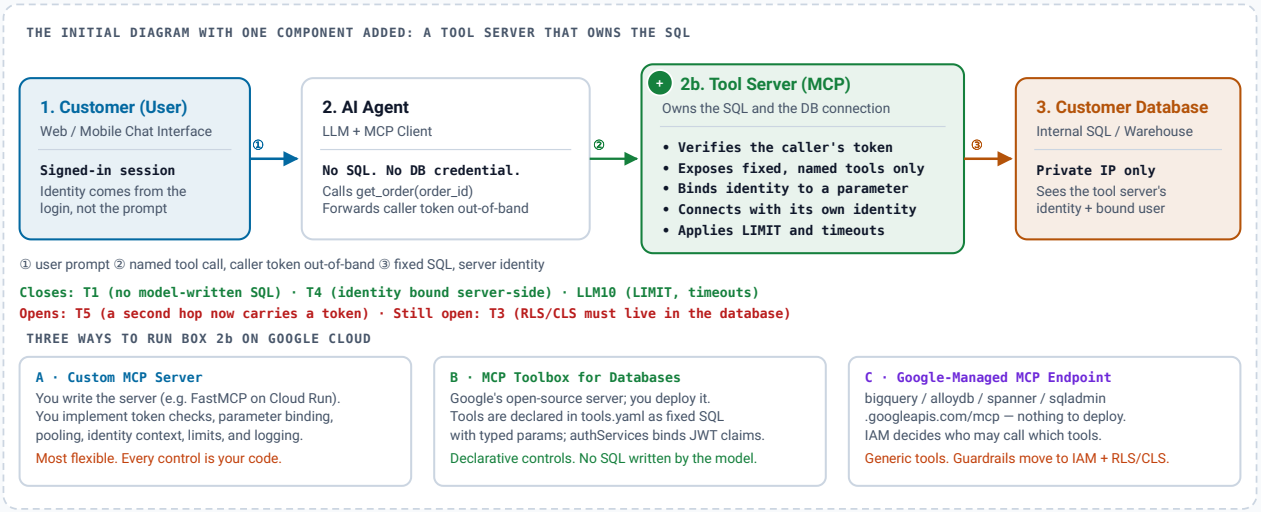


Figure R1 — The tool server inserted between the agent and the database, and the three ways to run it on Google Cloud.

▢ Added Component ■ + Newly Added

Option A · A Custom MCP Server

You write the server yourself—for example with `FastMCP` in Python, deployed on Cloud Run. This is the right choice when the tools are not simple SQL statements: multi-step business logic, calls to several systems, or a database that no off-the-shelf server supports. The cost is that every guardrail is your code. To close the threats listed in Episode 01 the server must, at minimum: verify the caller's token on every request and reject calls without one; expose only named tools whose SQL is fixed in code with bound parameters; take the user's identity from the verified token and never from a tool argument; connect to the database with the server's own IAM identity rather than a password; set the per-request identity context on the connection (for Row-Level Security) inside the same transaction as the query; enforce `LIMIT` and statement timeouts; and emit a trace ID with every call. None of this is difficult, but all of it is easy to omit, and an omission is invisible in a functional test.

Option B · MCP Toolbox for Databases

MCP Toolbox for Databases is an open-source MCP server from Google that you deploy yourself (Cloud Run or GKE) and configure declaratively. Instead of writing a server, you write a `tools.yaml` with three sections: `sources` (how to reach each database, using IAM authentication and a private IP), `tools` (each tool is one fixed SQL statement with typed parameters), and `authServices` (which identity provider to verify tokens against, and which token claim to bind to which parameter). The decisive feature is parameter binding from the token: a parameter declared under `authServices` is filled by the server from the verified JWT and is *not* exposed in the tool schema the model sees. The model therefore cannot choose whose data a query returns.

```yaml
tools.yaml — the controls that matter, shown together

sources:
  orders-db:
    kind: cloud-sql-postgres
    project: app-prod
    region: us-central1
    instance: orders-primary
    database: orders
    ipType: private            # no public IP path
    user: toolbox-sa@app-prod.iam   # IAM DB auth, no password

authServices:
  app_jwt:
    kind: google
    clientId: 1234567890-abc.apps.googleusercontent.com   # audience the token must carry

tools:
  get_order:
    kind: postgres-sql
    source: orders-db
    description: Return the status of one order that belongs to the signed-in customer.
    parameters:
      - name: owner_email
        type: string
        authServices:            # filled from the verified token, hidden from the model
          - name: app_jwt
            field: email
      - name: order_id
        type: string
    statement: |
      SELECT order_id, status, shipped_at
      FROM app.orders_v            -- a view without PII columns
      WHERE owner_email = $1 AND order_id = $2
      LIMIT 1;

toolsets:
  customer_support: [get_order]   # the agent is given this set, nothing else
```

With this configuration the agent sees one tool, `get_order(order_id)`. Its SQL cannot be changed from the prompt (closes `T1`), the owner is bound server-side from the token (closes `T4`), the view hides PII columns (part of `T2`), and the `LIMIT` bounds the result size (`LLM10`). What Toolbox does not do by itself is make the database enforce the filter: the `WHERE owner_email = $1` is still a clause in a statement, not an engine policy. If a

second tool is added later without that clause, the protection is gone. That is why the database stage of Episode 01 moves the filter into the database with Row-Level Security, keyed to the same bound identity.

Option C · Google-Managed MCP Endpoints

BigQuery, AlloyDB, Spanner, Cloud SQL, and Firestore each expose a Google-hosted MCP endpoint (for example `bigquery.googleapis.com/mcp`). There is nothing to deploy; access is controlled with IAM, and the database's own permissions apply to whatever identity makes the call. The trade-off is that the tools are generic, prebuilt primitives—for Cloud SQL, `execute_sql` and `execute_sql_readonly` (plus schema listing and user management)—so the server does not constrain what SQL the model writes. IAM and the database role restrict which tables are reachable; `execute_sql_readonly` blocks mutations; but a prompt-injected model can still compose any `SELECT` and read every row of every permitted table. Google's own guidance for the managed server is to use least-privilege database users and prefer the read-only tool (Secure agent interactions with the Cloud SQL MCP server); that reduces blast radius but does not change who writes the query. The guardrails therefore move entirely to the identity and the database: the call must be made with a credential scoped to the end user (an on-behalf-of token) or to a narrowly-permitted agent identity; IAM must restrict which tools that identity may invoke; and Row-Level and Column-Level Security in the database must do the filtering that a fixed statement would otherwise do. This option is well suited to analytical, read-only use over BigQuery where RLS and policy tags are already in place, and poorly suited to transactional writes.

	A · CUSTOM MCP SERVER	B · MCP TOOLBOX FOR DATABASES	C · GOOGLE-MANAGED MCP ENDPOINT
Who writes the SQL	You, in server code	You, in `tools.yaml` (fixed statements)	The model (generic execute-query tool)
How the user's identity reaches the DB	Your code: verify token → bind parameter or `set_config` in the transaction	`authServices` binds a JWT claim to a hidden parameter	The caller's credential itself (OBO token or agent identity); IAM + DB permissions
Credential used to connect	Server's IAM identity (must be configured; passwords are the default trap)	Server's IAM identity via `sources`	The caller's Google credential

	A · CUSTOM MCP SERVER	B · MCP TOOLBOX FOR DATABASES	C · GOOGLE-MANAGED MCP ENDPOINT
Configuration you must own	Token verification, schemas, binding, pooling, limits, timeouts, logging, deployment	`sources` (IAM, private IP), `authServices`, `tools` with `LIMIT`, `toolsets`, deployment identity	IAM roles per tool, per-user credentials, RLS/CLS in the database, VPC-SC support check
Threats closed by the layer itself	`T1` `T4` `LLM10` — only if implemented	`T1` `T4` `LLM10` by configuration	None directly; relies on `T3` controls in the DB
Threats that remain	`T2` `T3` `T5` `T6` `T7` `T8`	`T2` `T3` `T5` `T6` `T7` `T8`	`T1` `T2` `T3` `T5` `T6` `T7` `T8` `LLM10`
Best fit	Complex tools, non-SQL systems, bespoke logic	Transactional and analytical SQL with a bounded set of operations	Read-only analytics where IAM and RLS/CLS already govern access

Choosing the Approach

The database role is the ceiling: whatever it may read, a generic query tool lets the model reach on every call. The tool definition therefore has to be the floor. Exposing a generic `execute_sql` tool to an autonomous agent without a trusted human in the loop grants that agent read access to every row the database role permits—in a multi-tenant application, that is every tenant.

SITUATION	APPROACH	WHY
Autonomous agent, no trusted human in the loop, multi-tenant data	**Never** a generic `execute_sql` tool. Narrow, parameterized tools in MCP Toolbox `tools.yaml` (Option B), each returning a defined subset; or a custom server (A) with the same discipline.	The tool, not the prompt, is the only place the model cannot rewrite.
Human analyst in the loop, read-only, Row- and Column-Level Security already enforced by the warehouse	A generic query tool through the Google-managed MCP endpoint (Option C) is acceptable.	IAM and the database do the filtering; the human reviews before anything is acted on.
Bespoke logic or non-SQL systems	Custom server (A), still with narrow tools.	A custom server that exposes `run_sql` is Option C without IAM.

Whichever option is chosen, adding the tool server creates a new hop, and with it a new threat: `T5`. The agent now has to present some credential to the tool server. Forwarding the user's frontend token unchanged is the obvious shortcut and is explicitly forbidden by the MCP authorization specification, because a compromised tool server could replay it against every API the token was valid for. The correct pattern—exchanging the token for a

short-lived one scoped to the tool server—belongs to the identity layer and is covered in the next episode of the series.